

Rapport de TP — Implémentation d'un mini réseau de neurones pour le problème XOR

Auteur : ADIL HAMMAD

Formation : II BDCC

Date : 24 juin 2026

Résumé

Ce rapport présente la réalisation d'un mini réseau de neurones entraîné avec NumPy pour apprendre la fonction logique XOR. L'objectif principal est pédagogique : comprendre de bout en bout les briques fondamentales d'un réseau dense, depuis la propagation avant jusqu'à la rétropropagation et la mise à jour des paramètres par descente de gradient.

Le travail a été réalisé dans le notebook principal du projet. Il comprend deux volets :

- Le premier volet introduit et compare plusieurs fonctions d'activation (ReLU, Sigmoid, Tanh, Leaky ReLU, Softplus, ELU) afin de mettre en évidence leur comportement et leur impact potentiel sur l'apprentissage.
- Le second volet applique ces notions à un réseau $2 \rightarrow 3 \rightarrow 1$ pour résoudre XOR.

Les résultats montrent une convergence nette de la perte MSE, passant d'environ 0,2476 au début de l'entraînement à une valeur très faible après plusieurs milliers d'époques. Ce résultat confirme qu'une architecture non linéaire avec couche cachée permet de séparer un problème non linéairement séparable comme XOR. En complément, des visualisations de points, de convergence, de matrice de confusion et de courbe ROC permettent d'appuyer l'analyse.

1. Introduction

L'apprentissage automatique repose sur la capacité d'un modèle à ajuster des paramètres à partir de données. Le problème XOR est historiquement central car il met en défaut les modèles linéaires simples. En effet, les quatre points XOR ne peuvent pas être séparés par une seule droite dans le plan. Cette propriété rend le problème idéal pour introduire la notion de non-linéarité et la nécessité des couches cachées.

Dans ce TP, nous cherchons à répondre à trois questions :

- Comment construire un mini réseau de neurones dense uniquement avec NumPy ?

- Comment calculer les gradients à la main et implémenter la rétropropagation ?
- Comment vérifier objectivement que le modèle apprend correctement XOR ?

Le choix de rester proche des opérations matricielles fondamentales est volontaire. Il permet de comprendre ce que les bibliothèques de haut niveau automatisent d'habitude. Cette approche favorise une compréhension mécanique de la chaîne complète : passage avant, calcul de la perte, dérivées locales, règle de la chaîne, mise à jour des poids.

2. Contexte théorique

2.1. Définition de XOR

Pour deux entrées binaires x_1 et x_2 , la sortie XOR vaut 1 si les entrées sont différentes, et 0 sinon.

Table de vérité :

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Géométriquement, les classes positives occupent les deux coins opposés d'un carré, et les classes négatives les deux autres coins. Une séparation linéaire unique est impossible.

2.2. Architecture retenue

Le réseau utilisé est un Perceptron Multicouche (MLP) minimal :

- Entrée : 2 neurones
- Couche cachée : 3 neurones
- Sortie : 1 neurone

Cette architecture est notée $2 \rightarrow 3 \rightarrow 1$. La couche cachée introduit la capacité de représentation non linéaire. La sortie est sigmoïde pour produire une probabilité entre 0 et 1.

2.3. Équations du modèle

Soit X de dimension $(N, 2)$ et y de dimension $(N, 1)$.

Propagation avant :

$$Z_1 = XW_1 + b_1$$

$$A_1 = f(Z_1)$$

$$Z_2 = A_1W_2 + b_2$$

$$\hat{y} = g(Z_2)$$

Où f est l'activation de la couche cachée (souvent tanh) et g la sigmoïde en sortie.

Perte (MSE) :

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Rétropropagation :

$$dY = \frac{2}{N} (\hat{y} - y)$$

$$dZ_2 = dY \cdot g'(\hat{y})$$

$$dW_2 = A_1^T \cdot dZ_2$$

$$db_2 = \sum dZ_2$$

$$dA_1 = dZ_2 \cdot W_2^T$$

$$dZ_1 = dA_1 \cdot f'(Z_1)$$

$$dW_1 = X^T \cdot dZ_1$$

$$db_1 = \sum dZ_1$$

Mise à jour (Descente de gradient) :

$$W \leftarrow W - lr \cdot dW$$

$$b \leftarrow b - lr \cdot db$$

3. Mise en œuvre pratique

3.1. Environnement et outils

Le projet repose sur :

- Python
- NumPy pour le calcul vectoriel
- Matplotlib pour les visualisations
- SciPy pour des compléments de calcul et d'évaluation
- Notebook Jupyter comme support d'expérimentation

L'organisation du projet sépare clairement le notebook principal et les sorties graphiques dans le dossier reports.

3.2. Étape A — Étude des fonctions d'activation

Avant l'entraînement du réseau, plusieurs activations sont comparées sur un intervalle régulier de valeurs.

Fonctions analysées : ReLU, Sigmoid (et Sigmoid stable), Tanh, Leaky ReLU, Softplus, ELU.

Cette étape poursuit deux objectifs :

- Comprendre la forme de chaque activation et de sa dérivée.
- Anticiper les effets sur le gradient (saturation, zones mortes, pente locale).

Observations classiques :

Sigmoid et Tanh saturent aux extrémités, ce qui peut ralentir l'apprentissage. ReLU est efficace mais peut conduire à des neurones inactifs. Leaky ReLU et ELU atténuent ce risque en conservant une pente dans la zone négative. Softplus lisse le comportement de ReLU.

3.3. Étape B — Entraînement du réseau XOR

Le mini-dataset XOR est défini explicitement avec quatre points. Les paramètres W_1 , b_1 , W_2 et b_2 sont initialisés, puis le réseau est entraîné pendant plusieurs époques (de l'ordre de milliers d'itérations), avec un taux d'apprentissage élevé mais stable dans ce cadre simple.

La boucle d'entraînement effectue à chaque itération :

1. Forward pass (Propagation avant)
2. Calcul de la MSE

3. Backpropagation complète (Rétropropagation)
4. Mise à jour des paramètres
5. Journalisation périodique de la perte

Le notebook affiche des valeurs intermédiaires de perte, ce qui permet de vérifier rapidement que la descente s'effectue correctement.

4. Résultats expérimentaux

4.1. Convergence de la loss

Une valeur de loss initiale autour de 0,2476 est observée en début d'apprentissage. Puis la perte diminue de manière monotone et atteint des niveaux très faibles (de l'ordre de 10^{-3} , puis inférieurs), ce qui est cohérent avec un apprentissage réussi de XOR sur un petit dataset.

Interprétation :

- La pente est forte au début, signe d'une correction rapide des poids.
- La baisse devient plus lente en fin d'apprentissage, entrant dans un régime de convergence fine.
- L'absence d'oscillation importante suggère un taux d'apprentissage raisonnable pour ce cas.

4.2. Nuage de points XOR

Le nuage des quatre points confirme la structure non linéaire du problème :

- Les classes sont en diagonale opposée.
- Une frontière linéaire unique ne peut pas séparer les classes correctement.
- La couche cachée est indispensable pour construire une séparation en morceaux.

4.3. Comparaison des activations

Le notebook produit des graphiques comparatifs des activations, en figure unique et en sous-figures.

Analyse qualitative : Le couple Tanh (cachée) + Sigmoidé (sortie) est pertinent pour XOR. La sigmoïde stable est utile pour éviter des problèmes numériques en cas de grandes amplitudes. Les fonctions piecewise comme ReLU sont intéressantes, mais pour ce mini cas, Tanh reste très interprétable.

4.4. Mesures de classification

Le dossier contient également des visualisations de matrice de confusion et de courbe ROC. Sur un cas simple comme XOR, un modèle bien entraîné tend vers des prédictions quasi parfaites. Les deux graphes servent principalement à illustrer de bonnes pratiques d'évaluation et à préparer une extension vers des jeux de données plus réalistes.

5. Discussion

5.1. Pourquoi le réseau apprend-il XOR ?

Le réseau $2 \rightarrow 3 \rightarrow 1$ apprend XOR car la couche cachée transforme l'espace d'entrée en une représentation plus séparable. Chaque neurone caché peut être vu comme un détecteur partiel de région. La combinaison linéaire en sortie recompose ensuite la décision finale.

En d'autres termes, la non-linéarité de la couche cachée permet d'émuler une frontière de décision non linéaire, ce qu'un perceptron simple ne peut pas faire.

5.2. Limites de l'expérience

Malgré son intérêt pédagogique, ce TP reste limité :

- Taille de données très faible.
- Faible variabilité statistique.
- Sensibilité possible à l'initialisation.
- Résultats facilement surajustés (overfitting) au mini-jeu de points.

Ces limites sont normales pour un TP de base, mais doivent être explicitées pour éviter de généraliser à tort les conclusions.

5.3. Stabilité numérique

Le notebook montre une attention utile à la stabilité numérique, notamment avec `sigmoid_stable`. Cette précaution est importante car les exponentielles peuvent conduire à des phénomènes d'overflow ou d'underflow si les valeurs de Z deviennent trop grandes en amplitude.

Cette pratique est très recommandée pour les implémentations "maison", même sur des exemples simples.

6. Validation et interprétation des sorties

6.1. Vérifications structurelles

Les dimensions des matrices sont rigoureusement contrôlées :

Paramètre	Dimension
W_1	(2, 3)
b_1	(1, 3)
W_2	(3, 1)
b_2	(1, 1)

Ces vérifications évitent les erreurs silencieuses de broadcasting.

6.2. Lecture des prédictions

Pour valider la qualité du modèle, on observe $\hat{y}$ sur les quatre points XOR :

- Sorties proches de 0 pour les classes négatives.
- Sorties proches de 1 pour les classes positives.
- Un seuil de 0,5 permet ensuite la conversion en classes binaires.

6.3. Cohérence entre métriques et visualisations

Une bonne pratique de ce TP est la cohérence multi-niveaux :

- La perte baisse.
- Les prédictions deviennent correctes.
- La matrice de confusion est favorable.
- La courbe ROC est d'excellente qualité.

Cette convergence d'indices limite le risque d'une interprétation basée sur un seul indicateur.

7. Pistes d'amélioration

Plusieurs extensions peuvent enrichir ce travail :

- Comparer plusieurs initialisations aléatoires et tracer la variance de performance.
- Tester plusieurs taux d'apprentissage et analyser la stabilité.
- Remplacer la MSE par la Binary Cross-Entropy pour la sortie sigmoïde.

- Évaluer différents couples d'activations (cachée/sortie).
- Ajouter une régularisation L2 pour observer son effet.
- Introduire un XOR bruité de manière contrôlée.
- Mettre en place une validation croisée répétée sur des jeux synthétiques.
- Reproduire l'expérience dans un framework (PyTorch ou TensorFlow) et comparer les courbes.

Ces extensions permettraient de passer d'un TP introductif à une mini-étude expérimentale plus robuste.

8. Conclusion

Ce TP confirme de manière claire l'idée centrale suivante : une architecture linéaire ne suffit pas pour XOR, alors qu'un mini réseau à couche cachée et activation non linéaire apprend efficacement la tâche.

L'implémentation manuelle dans NumPy constitue un excellent exercice pour internaliser les mécanismes fondamentaux des réseaux de neurones. La baisse rapide et stable de la perte, combinée aux visualisations de classification, valide la cohérence de l'approche.

Au-delà du résultat sur XOR, la valeur pédagogique principale est méthodologique :

- Raisonner sur les dimensions (shapes) et les opérations matricielles.
- Relier la théorie au code.
- Vérifier systématiquement les sorties.
- Documenter les expériences par des graphes lisibles.